ALGOINVEST&TRADE

MARS 2026

OPTIMISATION D'UNE STRATÉGIE D'INVESTISSEMENT AVEC DES ALGORITHMES PYTHON

PRESENTÉ PAR
Réda Abdi

POUR
AlgoInvest&Trade

I - INTRODUCTION

a. La problématique

- **Contexte :** Maximiser le profit des clients de AlgoInvest&Trade
- **Objectif** : Créer un algorithme qui renvoie la meilleure sélection d’actions la plus rentable sous le budget imparti.

ALGOINVEST&TRADE

SOMMAIRE

I - Introduction 3

a. La problématique
b. Le problème du sac à dos

II - Solution brute-force 6

a. L'algorithme
b. Limites et transition vers une solution plus optimisé

III - Solution optimisé 8

a. La programation dynamique
b. L'algorithme dynamique en pseudocode

IV - Comparaison d'efficacité et de performances, avec Big-O 16

a. Définition notation Big-O
b. Brute-force vs version optimisé (programmation dynamique)

V - Algorithme vs humain : Comparaison sur des données antérieurs 19

a. Présentation des données
b. Rapports d'exploration

I - INTRODUCTION

a. La problématique

- **Action =**
  - un nom
  - un coût (en euros)
  - un bénéfice (en %)

- **Contraintes :**
  - Chaque action ne peut être acheté qu'une fois
  - L'action ne peut pas être acheté en fraction
  - Le budget est de 500 euros par client

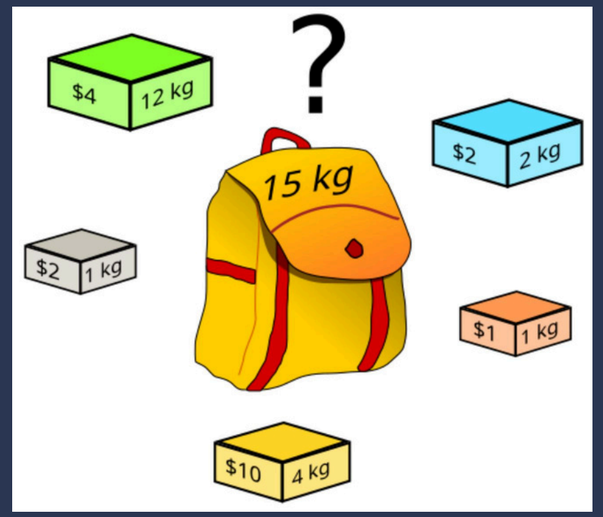
La liste d'action du moment :

Actions #	Coût par action (en euros)	Bénéfice (après 2 ans)
Action-1	20	5%
Action-2	30	10%
Action-3	50	15%
Action-4	70	20%
Action-5	60	17%
Action-6	80	25%
Action-7	22	7%
Action-8	26	11%
Action-9	48	13%
Action-10	34	27%
Action-11	42	17%
Action-12	110	9%
Action-13	38	23%
Action-14	14	1%
Action-15	18	3%
Action-16	8	8%
Action-17	4	12%
Action-18	10	14%
Action-19	24	21%
Action-20	114	18%

I - INTRODUCTION

b. Le problème du sac à dos

- **Définissons le problème du sac à dos**
- **Comment résoudre le problème**



II - SOLUTION BRUTE-FORCE

a. L'algorithme

- **Principe de la solution brute-force**
- **Algorithme en pseudocode :**

```
FONCTION brute_force(liste_actions):
  meilleure_combinaison = {combinaison_actions = [ ], cout_total = 0, benefice_total = 0}
  POUR combinaison_possible (liste_actions) :
    cout_total = 0
    benefice_total = 0
    POUR action DANS combinaison_possible:
      cout_total += action.cout
      benefice_total += action.benefice_euros
    SI cout_total <= 500 ET benefice_total > meilleure_combinaison["benefice_total"] :
      meilleure_combinaison["combinaison_actions"] = combinaison_possible
      meilleure_combinaison["cout_total"] = cout_total
      meilleure_combinaison["benefice_total"] = benefice_total

  RETOURNE meilleure_combinaison
```

II - SOLUTION BRUTE-FORCE

b. Limites et transition vers une solution optimisé

- **Limites**

- **Solutions optimisation ?**

Il existe differentes approche de résolution pour le problème du sac à dos, les plus utilisés sont :

- L'approche gloutonne : Rapide & Imprécis

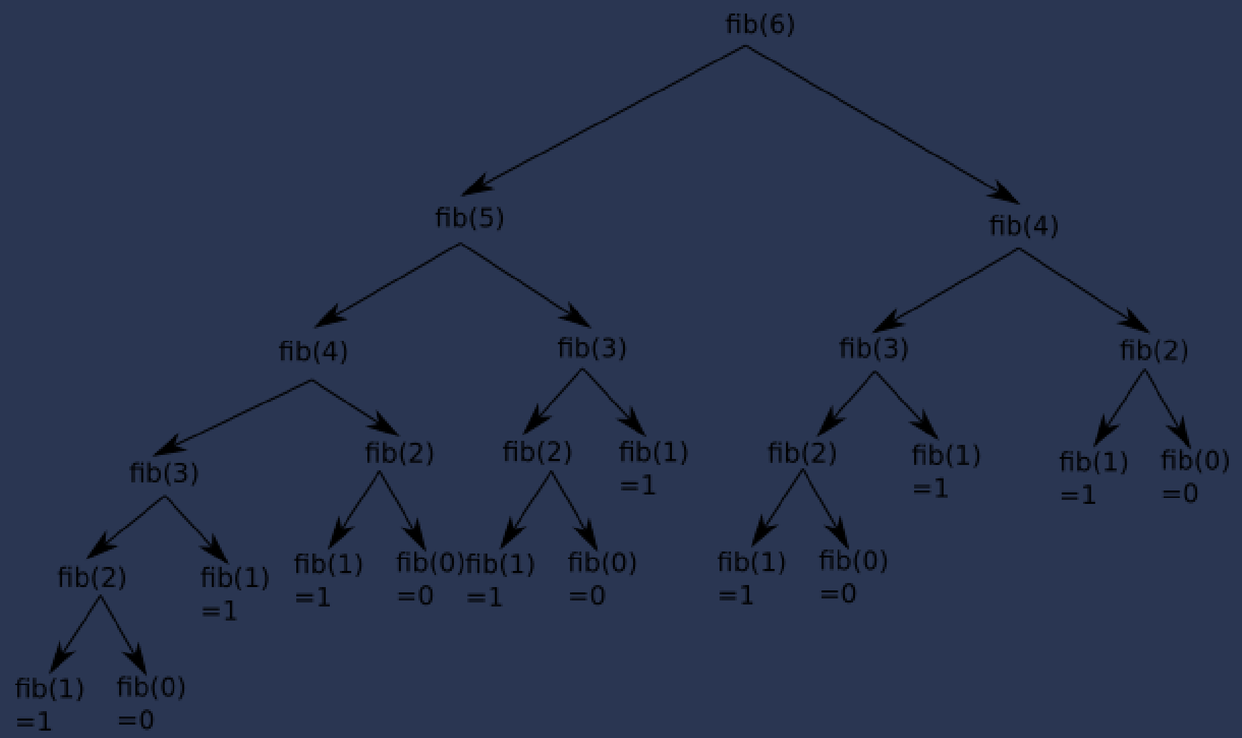
- La programmation dynamique : Précis & Gourmand

III - SOLUTION OPTIMISÉ

a. La programation dynamique

- **Qu’est ce que la programmation dynamique ?**

- **Problème de fibonacci :**



III - SOLUTION OPTIMISÉ

b. L’algorithme dynamique utilisé pour la solution optimisé

Etapes :

- 1) Création de la matrice
- 2) Remplissage de la matrice (Programmation dynamique)
- 3) Backtracking : Assigner la liste d’actions correspondant au bénéfices attribué à la dernière case

III - SOLUTION OPTIMISÉ

b. L'algorithme dynamique utilisé pour la solution optimisé

- **Paramètres**
- **1) Création de la matrice**

```
FONCTION programmation_dynamique (liste_actions, budget_max) :

  #Création de la matrice vide
  matrice = []
  POUR i allant à taille(liste_actions) + 1 :
      ligne = []
      POUR j allant à budget_max + 1 :
          ligne.ajouter(0)
      matrice.ajouter(ligne)
```

III - SOLUTION OPTIMISÉ

b. L’algorithme dynamique utilisé pour la solution optimisé

Budget : 6€
Nombre d’actions : 7

	0€	1€	2€	3€	4€	5€	6€
Pas d’action							
Action 1 Coût : 1€ Bénéfice : 4€							
Action 2 Coût : 2€ Bénéfice : 3€							
Action 3 Coût : 6€ Bénéfice : 10€							
Action 4 Coût : 3€ Bénéfice : 7€							
Action 5 Coût : 1€ Bénéfice : 3€							
Action 6 Coût : 2€ Bénéfice : 4€							
Action 7 Coût : 2€ Bénéfice : 6€							

III - SOLUTION OPTIMISÉ

b. L’algorithme dynamique utilisé pour la solution optimisé

2) Remplissage de la matrice

1) Est ce que l’action rentre dans le budget ?
Comparaison coût action et budget de la colonne

coût action > budget

Bénéfice max = action exclut

coût action < budget

2) Quelle est le meilleur bénéfice en incluant l’action obligatoirement

Calcul :
Bénéfice de l’action
+
Bénéfice sans l’action et sans coût action (budget - coût)

3) Comparaison action inclut vs action exclut

Bénéfice max = action exclut

Bénéfice max = action inclut

III - SOLUTION OPTIMISÉ

b. L'algorithme dynamique utilisé pour la solution optimisé

```
#Remplissage de la matrice
POUR action_index allant de 1 à taille(liste_actions) + 1 :
  action = liste_actions[action_index - 1]
  POUR budget allant de 1 à budget_max + 1 :
    SI action.coût <= budget :
      matrice[action_index][budget] = maximum(matrice[action_index - 1][budget],
      action.benefices + matrice[action_index - 1][budget-action.coût])
    SINON :
      matrice[action_index][budget] = matrice[action_index - 1][budget]
```

III - SOLUTION OPTIMISÉ

b. L’algorithme dynamique utilisé pour la solution optimisé

	0€	1€	2€	3€	4€	5€	6€
Pas d’action	0	0	0	0	0	0	0
Action 1 Coût : 1€ Bénéfice : 4€	0	4	4	4	4	4	4
Action 2 Coût : 2€ Bénéfice : 3€	0	4	4	7	7	7	7
Action 3 Coût : 6€ Bénéfice : 10€	0	4	4	7	7	7	10
Action 4 Coût : 3€ Bénéfice : 7€	0	4	4	7	11	11	14
Action 5 Coût : 1€ Bénéfice : 3€	0	4	7	7	11	14	14
Action 6 Coût : 2€ Bénéfice : 4€	0	4	7	11	11	14	15
Action 7 Coût : 2€ Bénéfice : 6€	0	4	7	11	13	17	17

III - SOLUTION OPTIMISÉ

b. L'algorithme dynamique utilisé pour la solution optimisé

- **Backtracking**

	0€	1€	2€	3€	4€	5€	6€
Pas d'action	0	0	0	0	0	0	0
Action 1 Coût : 1€ Bénéfice : 4€	0	4	4	4	4	4	4
Action 2 Coût : 2€ Bénéfice : 3€	0	4	4	7	7	7	7
Action 3 Coût : 6€ Bénéfice : 10€	0	4	4	7	7	7	10
Action 4 Coût : 3€ Bénéfice : 7€	0	4	4	7	11	11	14
Action 5 Coût : 1€ Bénéfice : 3€	0	4	7	7	11	14	14
Action 6 Coût : 2€ Bénéfice : 4€	0	4	7	11	11	14	15
Action 7 Coût : 2€ Bénéfice : 6€	0	4	7	11	13	17	17

III - SOLUTION OPTIMISÉ

b. L'algorithme dynamique utilisé pour la solution optimisé

- **Backtracking**

	0€	1€	2€	3€	4€	5€	6€
Pas d'action	0	0	0	0	0	0	0
Action 1 Coût : 1€ Bénéfice : 4€	0	4	4	4	4	4	4
Action 2 Coût : 2€ Bénéfice : 3€	0	4	4	7	7	7	7
Action 3 Coût : 6€ Bénéfice : 10€	0	4	4	7	7	7	10
Action 4 Coût : 3€ Bénéfice : 7€	0	4	4	7	11	11	14
Action 5 Coût : 1€ Bénéfice : 3€	0	4	7	7	11	14	14
Action 6 Coût : 2€ Bénéfice : 4€	0	4	7	11	11	14	15
Action 7 Coût : 2€ Bénéfice : 6€	0	4	7	11	13	17	17

III - SOLUTION OPTIMISÉ

b. L'algorithme dynamique utilisé pour la solution optimisé

- **Backtracking**

	0€	1€	2€	3€	4€	5€	6€
Pas d'action	0	0	0	0	0	0	0
Action 1 Coût : 1€ Bénéfice : 4€	0	4	4	4	4	4	4
Action 2 Coût : 2€ Bénéfice : 3€	0	4	4	7	7	7	7
Action 3 Coût : 6€ Bénéfice : 10€	0	4	4	7	7	7	10
Action 4 Coût : 3€ Bénéfice : 7€	0	4	4	7	11	11	14
Action 5 Coût : 1€ Bénéfice : 3€	0	4	7	7	11	14	14
Action 6 Coût : 2€ Bénéfice : 4€	0	4	7	11	11	14	15
Action 7 Coût : 2€ Bénéfice : 6€	0	4	7	11	13	17	17

III - SOLUTION OPTIMISÉ

b. L'algorithme dynamique utilisé pour la solution optimisé

- **Backtracking**

	0€	1€	2€	3€	4€	5€	6€
Pas d'action	0	0	0	0	0	0	0
Action 1 Coût : 1€ Bénéfice : 4€	0	4	4	4	4	4	4
Action 2 Coût : 2€ Bénéfice : 3€	0	4	4	7	7	7	7
Action 3 Coût : 6€ Bénéfice : 10€	0	4	4	7	7	7	10
Action 4 Coût : 3€ Bénéfice : 7€	0	4	4	7	11	11	14
Action 5 Coût : 1€ Bénéfice : 3€	0	4	7	7	11	14	14
Action 6 Coût : 2€ Bénéfice : 4€	0	4	7	11	11	14	15
Action 7 Coût : 2€ Bénéfice : 6€	0	4	7	11	13	17	17

III - SOLUTION OPTIMISÉ

b. L'algorithme dynamique utilisé pour la solution optimisé

- **Backtracking**

	0€	1€	2€	3€	4€	5€	6€
Pas d'action	0	0	0	0	0	0	0
Action 1 Coût : 1€ Bénéfice : 4€	0	4	4	4	4	4	4
Action 2 Coût : 2€ Bénéfice : 3€	0	4	4	7	7	7	7
Action 3 Coût : 6€ Bénéfice : 10€	0	4	4	7	7	7	10
Action 4 Coût : 3€ Bénéfice : 7€	0	4	4	7	11	11	14
Action 5 Coût : 1€ Bénéfice : 3€	0	4	7	7	11	14	14
Action 6 Coût : 2€ Bénéfice : 4€	0	4	7	11	11	14	15
Action 7 Coût : 2€ Bénéfice : 6€	0	4	7	11	13	17	17

III - SOLUTION OPTIMISÉ

b. L'algorithme dynamique utilisé pour la solution optimisé

- **Backtracking**

	0€	1€	2€	3€	4€	5€	6€
Pas d'action	0	0	0	0	0	0	0
Action 1 Coût : 1€ Bénéfice : 4€	0	4	4	4	4	4	4
Action 2 Coût : 2€ Bénéfice : 3€	0	4	4	7	7	7	7
Action 3 Coût : 6€ Bénéfice : 10€	0	4	4	7	7	7	10
Action 4 Coût : 3€ Bénéfice : 7€	0	4	4	7	11	11	14
Action 5 Coût : 1€ Bénéfice : 3€	0	4	7	7	11	14	14
Action 6 Coût : 2€ Bénéfice : 4€	0	4	7	11	11	14	15
Action 7 Coût : 2€ Bénéfice : 6€	0	4	7	11	13	17	17

III - SOLUTION OPTIMISÉ

b. L'algorithme dynamique utilisé pour la solution optimisé

- **Backtracking**

	0€	1€	2€	3€	4€	5€	6€
Pas d'action	0	0	0	0	0	0	0
Action 1 Coût : 1€ Bénéfice : 4€	0	4	4	4	4	4	4
Action 2 Coût : 2€ Bénéfice : 3€	0	4	4	7	7	7	7
Action 3 Coût : 6€ Bénéfice : 10€	0	4	4	7	7	7	10
Action 4 Coût : 3€ Bénéfice : 7€	0	4	4	7	11	11	14
Action 5 Coût : 1€ Bénéfice : 3€	0	4	7	7	11	14	14
Action 6 Coût : 2€ Bénéfice : 4€	0	4	7	11	11	14	15
Action 7 Coût : 2€ Bénéfice : 6€	0	4	7	11	13	17	17

III - SOLUTION OPTIMISÉ

b. L'algorithme dynamique utilisé pour la solution optimisé

- **Backtracking**

	0€	1€	2€	3€	4€	5€	6€
Pas d'action	0	0	0	0	0	0	0
Action 1 Coût : 1€ Bénéfice : 4€	0	4	4	4	4	4	4
Action 2 Coût : 2€ Bénéfice : 3€	0	4	4	7	7	7	7
Action 3 Coût : 6€ Bénéfice : 10€	0	4	4	7	7	7	10
Action 4 Coût : 3€ Bénéfice : 7€	0	4	4	7	11	11	14
Action 5 Coût : 1€ Bénéfice : 3€	0	4	7	7	11	14	14
Action 6 Coût : 2€ Bénéfice : 4€	0	4	7	11	11	14	15
Action 7 Coût : 2€ Bénéfice : 6€	0	4	7	11	13	17	17

III - SOLUTION OPTIMISÉ

b. L’algorithme dynamique utilisé pour la solution optimisé

- **Partie Backtracking en pseudocode**

```
#Backtracking : Assigner une liste d'actions à la dernière case
liste_actions_optimale = []
budget_actuel = budget_max
index_action = taille(liste_actions)
TANT QUE budget_actuel > 0 ET index_action > 0 :
  action = liste_actions[index_action - 1]
  SI matrice[index_action][budget_actuel] != matrice[index_action][budget_actuel]
    liste_actions_optimale.ajouter(action)
    budget_actuel -= action.cout
  index_action -= 1
```

III - SOLUTION OPTIMISÉ

b. L’algorithme dynamique utilisé pour la solution optimisé

- **Cas limites**

1) Nouvelles contraintes

2) Type de Knapsack

3) Trop grandes data

IV - COMPARAISON EFFICACITÉ ET PERFORMANCES AVEC BIG-O

b. Brute-force vs version optimisé (programmation dynamique)

- **Définition**
- **Comparaison de la complexité temporelle avec Big-O**

Pour n = taille de liste_actions

2^n

n

```
FONCTION brute_force(liste_actions):
  meilleure_combinaison = {combinaison_actions = [ ],
  cout_total = 0,benefice_total = 0}
  POUR combinaison_possible (liste_actions) :
    cout_total = 0
    benefice_total = 0
    POUR action DANS combinaison_possible:
      cout_total += action.cout
      benefice_total += action.benefice_euros
    SI cout_total <= 500 ET benefice_total > meilleure_combinaison
    ["benefice_total"] :
      meilleure_combinaison["combinaison_actions"] = combinaison_po
      ssible
      meilleure_combinaison["cout_total"] = cout_total
      meilleure_combinaison["benefice_total"] = benefice_total

  RETOURNE meilleure_combinaison
```

Complexité temporelle = $O(n \times 2^n)$

IV - COMPARAISON EFFICACITÉ ET PERFORMANCES AVEC BIG-O

b. Brute-force vs version optimisé (programmation dynamique)

- **Comparaison de la complexité temporelle avec Big-O**

Pour n = taille de liste_actions
Pour W = budget

```
FONCTION programmation_dynamique (liste_actions, budget_max) :

  #Création de la matrice vide
  matrice = []
  POUR i allant à taille(liste_actions) + 1 :
      ligne = []
      POUR j allant à budget_max + 1 :
          ligne.ajouter(0)
      matrice.ajouter(ligne)

  #Remplissage de la matrice
  POUR action_index allant de 1 à taille(liste_actions) + 1 :
    action = liste_actions[action_index - 1]
    POUR budget allant de 1 à budget_max + 1 :
      SI action.coût <= budget :
        matrice[action_index][budget] = maximum(matrice[acti
        on_index - 1][budget],action.benefices + matrice[act
        ion_index - 1][budget-action.coût])
      SINON :
        matrice[action_index][budget] = matrice[action_inde
        x - 1][budget]

  #Backtracking : Assigner une liste d'actions à la dernière case
  liste_actions_optimale = []
  budget_actuel = budget_max
  index_action = taille(liste_actions)
  TANT QUE budget_actuel > 0 ET index_action > 0 :
    action = liste_actions[index_action - 1]
    SI matrice[index_action][budget_actuel] == action.benefices +
    matrice[index_action-1]
    [budget_actuel - action.cout] :
      liste_actions_optimale.ajouter(action)
      budget_actuel -= action.cout
    index_action -= 1
```

n × W

n × W

n

Complexité temporelle de O(n x W + n x W + n)

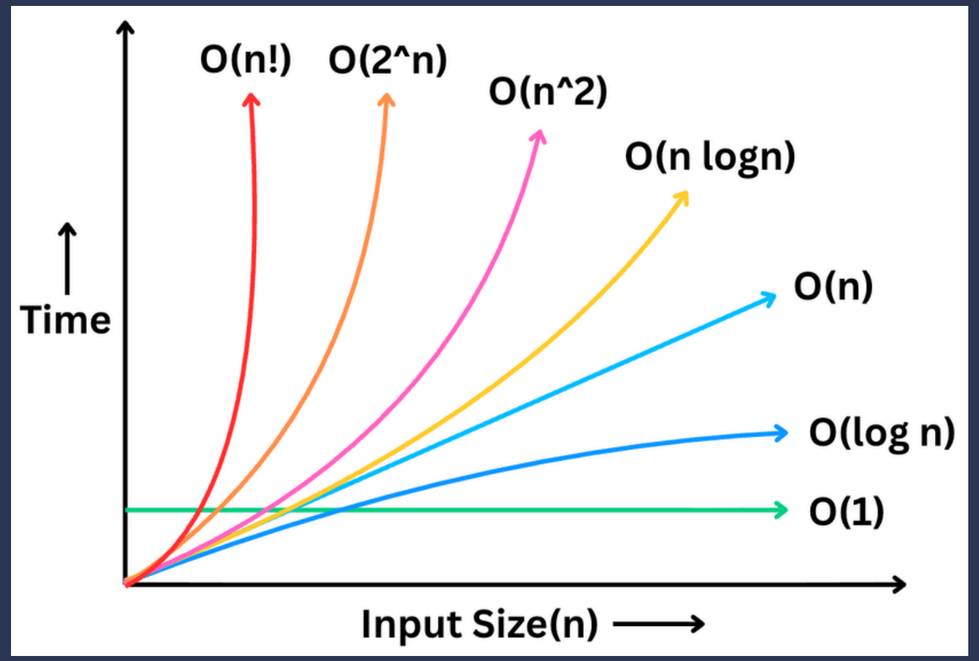
≃

O(n x W)

IV - COMPARAISON EFFICACITÉ ET PERFORMANCES AVEC BIG-O

b. Brute-force vs version optimisé (programmation dynamique)

- **Comparaison de la complexité temporelle avec Big-O**



Solution brute_force → une complexité temporelle exponentielle
Solution optimisé → temps d'exécution polynomiale

IV - COMPARAISON EFFICACITÉ ET PERFORMANCES AVEC BIG-O

b. Brute-force vs version optimisé (programmation dynamique)

- **Analyse de l'utilisation mémoire avec Big- O**

Solution brute-force = $O(1) + O(n) \simeq O(n)$
Solution optimisé : $O(1)$ + matrice $n \times W \simeq O(n \times W)$

- **Conclusion**

V - ALGORITHME VS HUMAIN : COMPARAISON SUR DES DONNÉES ANTÉRIEURS

a. Présentation des données

- **Présentation des fichiers**

Extrait dataset1.csv

1	name	price	profit
2	Share-DUPH	10.01	12.25
3	Share-GTAN	26.04	38.06
4	Share-USUF	9.25	27.69
5	Share-CFOZ	10.64	38.21
6	Share-QLRX	15.72	27.47
7	Share-XJDT	0.0	37.11
8	Share-HKFP	23.97	19.66
9	Share-PPPH	24.06	38.2
10	Share-HLJY	26.98	5.54

Extrait dataset2.csv

1	name	price	profit
2	Share-MOEX	40.6	16.69
3	Share-GBGY	27.08	34.09
4	Share-AXWK	-9.27	27.19
5	Share-FJTI	33.5	20.81
6	Share-LGDP	15.26	3.4
7	Share-GEBJ	5.87	37.95
8	Share-GGNT	33.39	7.88
9	Share-FDLS	-6.38	10.96
10	Share-JDYS	-2.7	11.96

V - ALGORITHME VS HUMAIN : COMPARAISON SUR DES DONNÉES ANTÉRIEURS

a. Présentation des données

- **Présentation des fichiers**

Sienna décision d'achats Dataset 1

```
Sienna bought:

Share-GRUT

Total cost: 498.76€
Total return: 196.61€
```

Sienna décision d'achats Dataset 2

```
Sienna bought:
Share-ECAQ 3166
Share-IXCI 2632
Share-FWBE 1830
Share-ZOFA 2532
Share-PLLK 1994
Share-YFVZ 2255
Share-ANFX 3854
Share-PATS 2770
Share-NDKR 3306
Share-ALIY 2908
Share-JWGF 4869
Share-JGTW 3529
Share-FAPS 3257
Share-VCAX 2742
Share-LFXB 1483
Share-DWSK 2949
Share-XQII 1342
Share-ROOM 1506

Total cost: 489.24€
Profit: 193.78€
```

Algorithme décision d'achats Dataset 1

```
> Share-QQTU :
  Cost : 33.19€
  Benefits after 2 years : 13.14€(39.6%)

> Share-GIAJ :
  Cost : 10.75€
  Benefits after 2 years : 4.29€(39.9%)

> Share-XJMO :
  Cost : 9.39€
  Benefits after 2 years : 3.75€(39.98%)

> Share-LRBZ :
  Cost : 32.9€
  Benefits after 2 years : 13.14€(39.95%)

> Share-KZBL :
  Cost : 28.99€
  Benefits after 2 years : 11.35€(39.14%)

> Share-EMOV :
  Cost : 8.89€
  Benefits after 2 years : 3.51€(39.52%)

> Share-IFCP :
  Cost : 29.23€
  Benefits after 2 years : 11.66€(39.88%)

Total cost : 499.96€
Total benefits (after 2 years) : 198.55€

Time : 12.169178247451782 seconds
```

Algorithme décision d'achats Dataset 2

```
> Share-JGTW :
  Cost : 35.29€
  Benefits after 2 years : 13.91€(39.43%)

> Share-FAPS :
  Cost : 32.57€
  Benefits after 2 years : 12.88€(39.54%)

> Share-VCAX :
  Cost : 27.42€
  Benefits after 2 years : 10.69€(38.99%)

> Share-LFXB :
  Cost : 14.83€
  Benefits after 2 years : 5.9€(39.79%)

> Share-DWSK :
  Cost : 29.49€
  Benefits after 2 years : 11.6€(39.35%)

> Share-XQII :
  Cost : 13.42€
  Benefits after 2 years : 5.3€(39.51%)

> Share-ROOM :
  Cost : 15.06€
  Benefits after 2 years : 5.91€(39.23%)

Total cost : 499.92€
Total benefits (after 2 years) : 197.96€

Time : 6.608839750289917 seconds
```

V - ALGORITHME VS HUMAIN : COMPARAISON SUR DES DONNÉES ANTÉRIEURS

b. Rapports d’exploration

- **Contexte**
- **Rapport d'exploration dataset 1 & 2**

	Nombre d’actions	Nombre d’actions erronés (0 ou négatif)	Coût min	Coût max	Coût moy	Coût median	Coût écart type	Bénefice min	Bénefice max	Bénefice moy	Bénefice median	Bénefice ecart-type
DATASET 1	1001	44	0.01€	498.76€	25.54€	25.62€	19.37 €	0.02%	39.98%	20.23%	19.82%	11.48%
DATASET 2	1000	459	0.12€	51.46€	25.61€	26.12€	10.50 €	0.15%	39.97%	19.57%	19.71%	12.20%

V - ALGORITHME VS HUMAIN : COMPARAISON SUR DES DONNÉES ANTÉRIEURS

b. Rapports d'exploration

- **Rapport d'exploration choix de Sienna vs choix de l'algorithme**

DATASET 1	Sienna	Algorithme
Coût total	498,76€	499,96€
Bénéfice net	196,61€	198,55€
Bénéfice en %	+39,42%	+39,71%
Temps d'execution	X	≃ 8 secondes

DATASET 2	Sienna	Algorithme
Coût total	489,24€	499,92€
Bénéfice net	193,78€	197,96
Bénéfice en %	+39,6%	+39,59%
Temps d'execution	X	≃ 5 secondes

ALGOINVEST&TRADE

MERCI DE VOTRE TEMPS

Email

redaabdi@algoinvestandtrade.com

Réseaux

linkedin.com/in/redaabdi/

Contact téléphonique

07 82 88 37 12